開発者、セキュリティ担当者も喜ぶ

Nix Flakeの優位性

Ryosuke Tomita

このスライドについて

このスライドは **AI (Claude) と一緒に作成しました。**
内容については作成者が精査・確認しています。

Nixファミリー全体像

コンポーネント	役割	今日話す?
Nix言語	純粋関数型の設定記述言語	少し触れる
Nixpkgs	10万以上のパッケージ集	—
NixOS	Nix言語でOSごと宣言的管理するLinuxディストロ	—
nix-shell	一時的な開発環境を作る旧来のコマンド	—
⭐ **Nix Flake**	依存をロックし再現性を保証する新しい仕組み	**← ここ**
Home Manager	ドットファイル・ユーザー環境をNixで管理	—

> Nix Flake は「Nixエコシステムの中で依存管理・再現性を担う層」

Nix Flakeとは

Nixパッケージマネージャーに追加された**実験的新機能** (Nix 2.4 / 2021年11月〜)

> **Nix言語**で開発環境・ビルド・依存関係をすべて宣言的に管理する仕組み

体感だが、2025年秋くらいからAIありきなら学習コストが高くないため、流行ってきているイメージ

3つの特徴

#	特徴	概要
1	**再現性**	同じ入力 → 誰でも・どこでも同じ出力
2	**宣言的**	Nix言語で環境・依存をコードとして記述
3	**信頼性**	フルコミットハッシュで固定、意図しない変更を構造的にブロック

Nix言語は純粋関数型で設定を記述できる

純粋関数とは?

> 同じ引数を渡せば、**必ず同じ結果**を返す関数
> 外部の状態に依存せず、副作用を持たない

```nix
# flake.nix の例
{
  description = "Haskell";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
    flake-utils.url = "github:numtide/flake-utils";
  };

  outputs =
    {
      self,
      nixpkgs,
      flake-utils,
    }:
    flake-utils.lib.eachDefaultSystem (
      system:
      let
        pkgs = import nixpkgs { inherit system; };
      in
      {
        formatter = nixpkgs.legacyPackages.${system}.nixfmt-tree;

        devShells.default = pkgs.mkShell {
          packages = [
            pkgs.zsh
            (pkgs.haskell.packages.ghc96.ghcWithPackages (ps: [
              ps.vector
              ps.containers
              ps.bytestring
            ]))
            pkgs.haskell.packages.ghc96.cabal-install
            pkgs.haskell.packages.ghc96.haskell-language-server
          ];
```

開発者的に嬉しい点

自分の開発環境だけ動かないがなくなる

新規メンバーが開発環境をつくるとうまくいかない...

Nixの解決策: フルコミットハッシュで管理

`flake.lock` が依存関係を**完全に固定**する

```
{
  "nodes": {
    "flake-utils": {
      "inputs": {
        "systems": "systems"
      },
      "locked": {
        "lastModified": 1731533236,
        "narHash": "sha256-l0KFg5HjrsfsO/JpG+r7fRrqm12kzFHyUHqHCVpMMbI=",
        "owner": "numtide",
        "repo": "flake-utils",
        "rev": "11707dc2f618dd54ca8739b309ec4fc024de578b",
        "type": "github"
      },
```

Dev Containers(Docker)を使った開発環境共通化との比較

評価観点	Docker	Nix Flake
環境の再現性	△	◎ (ハッシュで完全固定)
セキュリティ	△ (タグはmutable)	◎ (SHAで不変)
ホストOSへの影響	なし (コンテナ分離)	最小 (/nix/store に隔離)
CI/CD連携	◎	◎
Dockerイメージ生成	◎	◎ (Nixから生成可能!)

> `pkgs.dockerTools.buildImage` でNix管理の依存を使った
> **軽量・再現性の高いDockerイメージ**を生成できる

Nix Flakeのほうがコンテナ起動しなくていいのでストレスが減った。

Dockerより依存関係を追いやすい

```
which ghc
/nix/store/m1mjv78p7ah8yzc6h31vynli0s61dard-ghc-9.6.7/bin/ghc
sigma@lifebook:~/quine-haskell (main) $ nix-store -q --tree $(which ghc) | head -n 10
/nix/store/m1mjv78p7ah8yzc6h31vynli0s61dard-ghc-9.6.7
├───/nix/store/wb6rhpznjfczwlwx23zmdrrw74bayxw4-glibc-2.42-47
│   ├───/nix/store/d0d9wqmw5saaynfvmszsda3dmh5q82z8-libidn2-2.3.8
│   │   ├───/nix/store/pkphs076yz5ajnqczzj0588n6miph269-libunistring-1.4.1
│   │   │   └───/nix/store/pkphs076yz5ajnqczzj0588n6miph269-libunistring-1.4.1 [...]
│   │   └───/nix/store/d0d9wqmw5saaynfvmszsda3dmh5q82z8-libidn2-2.3.8 [...]
│   ├───/nix/store/kbijm6lc9va8xann3cfyam0vczzmwkxj-xgcc-15.2.0-libgcc
│   └───/nix/store/wb6rhpznjfczwlwx23zmdrrw74bayxw4-glibc-2.42-47 [...]
├───/nix/store/f15k3dpilmiyv6zgpib289rnjykgr1r4-bash-5.3p9
│   ├───/nix/store/wb6rhpznjfczwlwx23zmdrrw74bayxw4-glibc-2.42-47 [...]
```

観点	Docker	Nix
依存の一覧取得	`docker inspect` で間接的	`nix-store -q --references` で直接確認
依存ツリーの可視化	困難	`nix-store -q --tree` で木構造表示
不要な依存の検出	困難	`nix-store --gc` で未参照を自動検出

サプライチェーンセキュリティ対策になるかも(?)

サプライチェーン攻撃の現実: TeamPCP事件 (2026年)

"When the Security Scanner Became the Weapon"

(出典: SANS Webcast)

```
Day 0 (Feb 28): hackerbot-claw が Trivy の GitHub Actions 設定ミスを発見
                → pull_request_target 脆弱性で aqua-bot PAT を窃取

Day 1 (Mar 19): 76/77 の trivy-action タグを悪意あるコミットに force push
                → バックドア入り Trivy v0.69.4 を Docker Hub / ECR 等に配布

Day 2〜6:       盗んだクレデンシャルで連鎖的に拡散
```

被害	規模
npm (CanisterWorm)	66+ パッケージに悪意あるコード
Checkmarx KICS/AST	35 タグが侵害
LiteLLM (PyPI)	日次3.6M DLのパッケージが約3時間汚染
Docker Hub	v0.69.5/0.69.6 が配布

Nix Flakeなら意図しない更新は走らない

TeamPCP攻撃との対比

```
uses: aquasecurity/trivy-action@v0.24.0 # アンチパターン
uses: aquasecurity/trivy-action@57a97c7e7821...
```

Nix Flakeは最初からこの設計

```
# flake.lock に記録されるのは常にフルコミットハッシュ
"trivy": {
  "locked": {
    "rev": "57a97c7e7821a0a2b79bdfc4b78a45f6d5b9a1c3"
  }
}
```

- `nix flake update` を**明示的に実行しない限り**依存は変わらない
- **意図しないサプライチェーン汚染を構造的にブロック**
  - (フルコミットハッシュがすでにマリシャスで無い場合)

まとめ

- Nix Flakeはフルコミットハッシュを使いすべての依存を管理する
  - 再現性の高い開発環境を提供できる。
  - tagを侵害されても(コミットされた時点でマリシャスなコードでなければ)影響をうけない。

学習コストは高そうに見えるがAIに書いてもらえば問題ない!!素晴らしい時代!

実用Tips: direnv連携 & nix fmt

前提: nix develop — flake.nixで定義した環境に入るコマンド

```
nix develop     # flake.nix の devShells に定義したツール群が使える状態になる
ghc --version   # → Nixで固定したGHCが使える
exit            # → 環境を抜ける
```

direnv — ディレクトリに入るだけで自動で `nix develop` できる

```
# .envrc に1行書くだけ
echo "use flake" > .envrc && direnv allow
cd myproject    # → 自動でGHCやツールがPATHに乗る
cd ..           # → 環境がリセットされる
```

nix fmt — 複数言語を一括フォーマット

```
nix fmt   # Haskell / Nix / YAML など flake.nixで設定した言語を一発フォーマット
```

参考文献

- SANS Webcast: "When the Security Scanner Became the Weapon"
- NixOS Wiki: Flakes
- Nix 2.4 Release Notes (2021-11-01)